

Documentação Técnica

Simulador do Modelo OSI

Comunicação de Dados — Prof. Vinícius S. Borges — Semestre 2026/2 — Grupo 1

Eduardo Souza Urbanovicz Bastiani (082230018) · Ronaldo de Oliveira Santos (082230031) · João Vitor Maciel Nai (082230004)

1. Visão geral

O simulador implementa o modelo OSI de sete camadas sobre uma rede com três redes locais, quatro roteadores e cinco computadores. Uma mensagem gerada na camada 7 de um computador desce a pilha, atravessa a rede e sobe a pilha do computador de destino, e cada ação de cada camada produz uma linha de registro e uma representação visual da unidade de dados naquele instante.

Computadores implementam as sete camadas. Roteadores implementam apenas as três primeiras: o quadro sobe até a camada 3, onde a rota é decidida, e desce novamente em um quadro novo.

O programa opera em dois modos sobre o mesmo núcleo. O modo gráfico apresenta o mapa da rede, as pilhas, a unidade de dados desenhada e o registro; o modo texto percorre os mesmos cenários por um menu de terminal. A interface consome uma lista de eventos produzida pelo núcleo e não conhece a implementação das camadas.

2. Separação entre as camadas

Este é o eixo do projeto e determina a estrutura de todo o código.

A interface entre camadas adjacentes

Cada camada é uma classe com exatamente dois métodos de sentido oposto:

- `descer(pdu, contexto)` — recebe a unidade de dados da camada superior, acrescenta o que lhe compete e devolve o resultado para a camada inferior.
- `subir(pdu, contexto)` — recebe a unidade da camada inferior, remove o que lhe compete e devolve o resultado para a camada superior.

Ambos devolvem um par: uma tupla de unidades de dados e uma lista de eventos de registro. A tupla existe porque nem toda passagem por uma camada produz exatamente uma unidade: a camada 4 devolve vários segmentos ao segmentar, e devolve a tupla vazia quando um quadro é descartado ou quando ainda aguarda os demais segmentos. A camada não precisa de casos especiais para sinalizar essas situações.

Como se garante que nenhuma camada acessa uma camada não adjacente

A garantia não depende de disciplina de quem escreve o código. Ela é estrutural, e se apoia em três mecanismos.

Nenhuma camada guarda referência a outra camada. Uma classe de camada não possui atributo que aponte para a camada acima ou abaixo. Quem encadeia é a pilha do dispositivo, em `dispositivos.py`, que percorre a lista de camadas na ordem devida e passa a unidade de uma para a seguinte. Uma camada não tem como chamar outra porque não tem como alcançá-la.

A leitura de cabeçalho é restrita ao próprio número. Toda consulta a cabeçalho passa por um de dois métodos, e ambos leem sempre o número da própria camada: `Camada._meu_cabecalho`, na classe base, para os cabeçalhos, e `Enlace._meu_finalizador`, que existe porque o primeiro ignora finalizadores e a camada 2 precisa ler o seu para conferir a soma. Nenhum dos dois recebe a camada como argumento — cada um a toma de `self.numero`. Uma camada não lê o cabeçalho de outra porque nenhum dos caminhos disponíveis permite nomear outro número. Isso implementa a restrição de que o cabeçalho inserido por uma camada só é removido e interpretado pela camada de mesmo número no outro extremo.

A decisão de rota não atravessa camadas. A camada 3 grava a interface de saída e o próximo salto no contexto compartilhado. A camada 2 lê esses valores e traduz o próximo salto em endereço físico. A camada 2 não consulta a tabela de encaminhamento e não escolhe caminho: ela entrega ao vizinho que a camada 3 indicou.

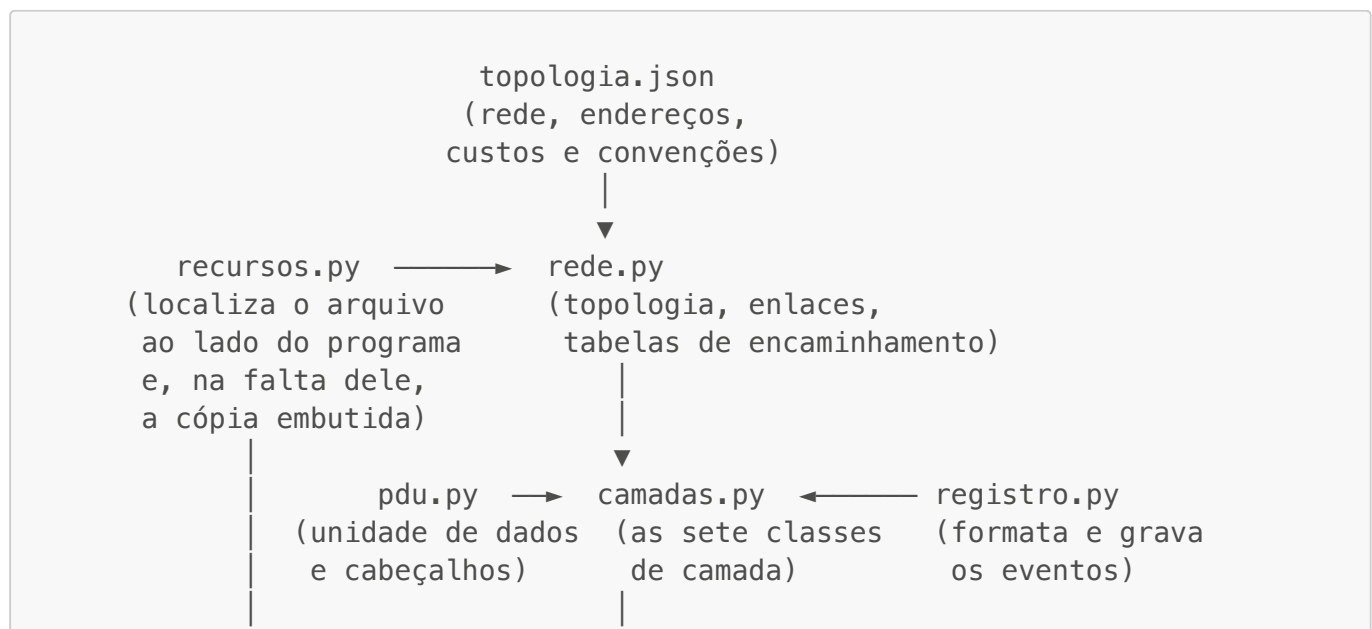
Por que o roteador não consegue ler uma porta

Porque não existe, dentro de um roteador, nenhum objeto capaz de fazê-lo.

A classe `Roteador` instancia apenas três camadas: rede, enlace e física. As camadas 4 a 7 não são construídas. A porta é um campo do cabeçalho da camada 4, e o único código que sabe interpretá-lo é a classe da camada 4 — que não está presente. Não há verificação em tempo de execução impedindo o acesso, nem convenção a ser respeitada: a capacidade simplesmente não foi montada.

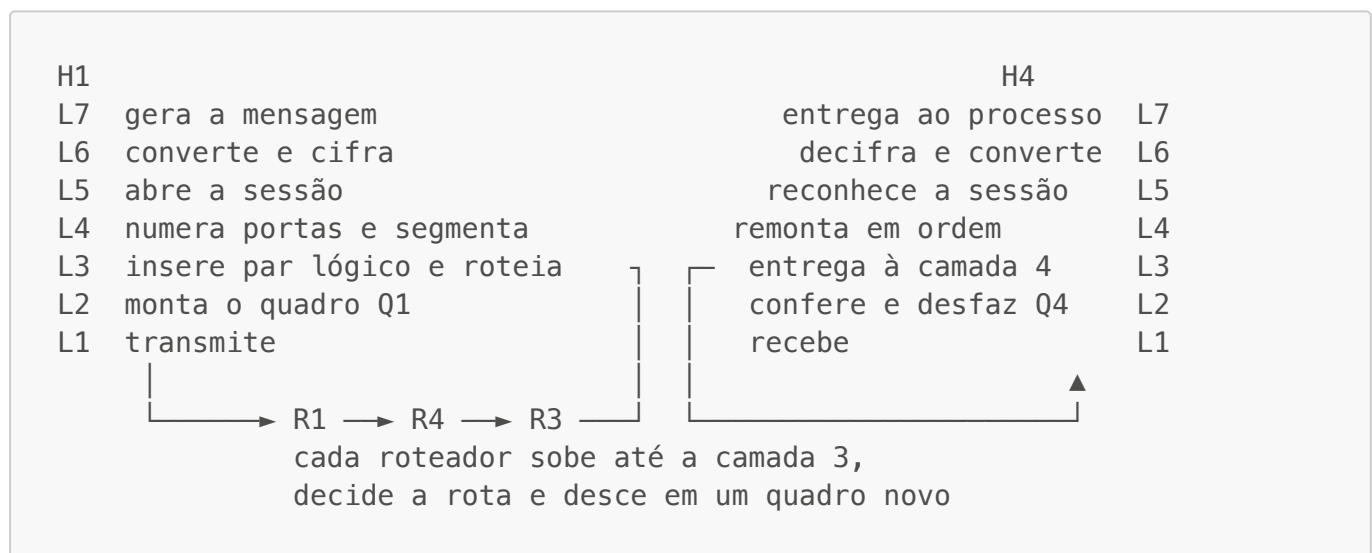
A consequência é que uma tentativa de fazer um roteador consultar uma porta não produz um valor errado, produz um erro de programa. O erro de modelagem se manifesta como falha, e não como resultado plausível.

3. Diagrama de módulos





Fluxo dos dados, da geração à entrega



O par de endereços lógicos inserido pela camada 3 de H1 permanece o mesmo nos quatro quadros. O par de endereços físicos é substituído em cada enlace.

4. Estrutura da unidade de dados

A unidade de dados é imutável. Encapsular e desencapsular não alteram o objeto recebido: devolvem um objeto novo. É isso que permite exibir quadros distintos em cada enlace, conforme a convenção de que o quadro recebido é descartado e um quadro novo é construído na saída, e é o que permite ao registro guardar o estado exato de cada passo.

Campo	Tipo	Função
dados	bytes ou texto	Conteúdo transportado
cabecalhos	tupla de Cabecalho	Cabeçalhos já acrescentados, mais o finalizador da camada 2

Campo	Tipo	Função
camada	inteiro	Camada em que a unidade se encontra
logicos	tupla	Par de endereços lógicos, de origem e destino
fisicos	tupla	Par de endereços físicos do salto corrente
portas	tupla	Par de portas de origem e destino
sessao	texto	Identificador da sessão
id_pacote	texto	Identifica o pacote ao longo de todo o percurso
numero_quadro	texto	Identifica o quadro em um único enlace
numero_segmento	inteiro	Posição do segmento na mensagem
total_segmentos	inteiro	Quantidade de segmentos da mensagem
cifrado	booleano	Indica se o conteúdo está cifrado

O nome da unidade — mensagem, segmento, pacote, quadro ou bits — é derivado da camada em que ela se encontra, e não armazenado.

O tamanho também é sempre derivado, nunca armazenado: resulta da soma dos dados com todos os cabeçalhos presentes. A conta exibida na tela e a figura desenhada não podem divergir, porque ambas vêm da mesma estrutura.

Cada cabeçalho é descrito por três atributos: a camada que o inseriu, o tamanho fixo em octetos e um conjunto de campos. O tamanho vem das convenções de simulação e não do conteúdo dos campos: os campos existem para exibição e para o registro, enquanto o tamanho é o que entra na conta de eficiência.

5. Interface dos módulos

pdu.py

Elemento	Recebe	Devolve	O que faz
Cabecalho	camada, tamanho, campos, finalizador	—	Descreve um cabeçalho ou o finalizador da camada 2
PDU.tamanho	—	inteiro	Octetos correntes: dados mais cabeçalhos
PDU.tamanho_em_bits	—	inteiro	Tamanho em bits, usado pela camada 1
PDU.nome_da_unidade	—	texto	Nome da unidade na camada corrente
PDU.encapsular	um cabeçalho, a camada de destino	PDU nova	Acrescenta um cabeçalho

Elemento	Recebe	Devolve	O que faz
<code>PDU.desencapsular</code>	número da camada	PDU nova	Remove os cabeçalhos daquela camada
<code>PDU.cabecalho_da_camada</code>	número da camada	Cabecalho ou nada	Lê o cabeçalho inserido pela camada par
<code>PDU.em_blocos</code>	—	lista	Descreve a unidade como blocos, da esquerda para a direita
<code>PDU.com_fisicos</code>	origem, destino, número do quadro	PDU nova	Constrói o quadro do salto seguinte

rede.py

Elemento	Recebe	Devolve	O que faz
<code>Topologia.carregar</code>	nome do arquivo	Topologia	Lê a rede do arquivo ao lado do programa ou, na falta dele, da cópia embutida, e guarda em .origem de qual das duas veio
<code>Topologia.mesma_rede</code>	dois endereços lógicos	booleano	Decide entre entrega direta e indireta
<code>Topologia.prefixo_de</code>	endereço lógico	prefixo ou nada	Devolve a rede do endereço; nada significa fora da topologia
<code>Topologia.tabela_de_encaminhamento</code>	nome do roteador	lista de Rota	Monta a tabela do roteador
<code>Topologia.rota_para</code>	roteador, endereço de destino	Rota ou nada	Consulta a rota; nada provoca o descarte na camada 3
<code>Topologia.caminho_de_dispositivos</code>	origem, destino	lista de nomes ou nada	Sequência percorrida, usada pelo mapa
<code>Topologia.derrubar</code>	dois dispositivos	booleano	Retira um enlace de serviço
<code>Topologia.vizinhos</code>	nome do roteador	lista	Roteadores adjacentes, com custo e interface

recursos.py

Elemento	Devolve	O que faz
<code>pasta_do_programa</code>	caminho	Pasta do executável quando empacotado, raiz do projeto quando executado do código-fonte
<code>pasta_embutida</code>	caminho ou nada	Pasta temporária em que o empacotador descompactou os dados embutidos; nada quando o programa roda do código-fonte
<code>caminho_de</code>	caminho	Monta o caminho de um arquivo ao lado do programa
<code>localizar</code>	Origem	Procura o arquivo ao lado do programa e, só então, na cópia embutida; é o que <code>rede.py</code> importa
Origem	—	De onde o arquivo veio: o caminho, se é a cópia embutida e onde ele foi procurado primeiro, para a tela informar ao usuário

camadas.py

Todos os métodos `descer` e `subir` recebem a unidade de dados e o contexto do dispositivo, e devolvem um par: uma tupla de unidades novas e uma lista de eventos. As colunas abaixo dizem, em cada caso, o que a unidade é na entrada e na saída.

Elemento	Recebe	Devolve	O que faz
Contexto	dispositivo, topologia e os contadores de passo, quadro e sessão	—	Reúne o que a pilha entrega a todas as camadas do dispositivo: processo, par de portas, endereço de destino, processos que escutam cada porta, interface de saída e próximo salto
<code>ContadorDeQuadros.proximo</code>	par de endereços lógicos	inteiro	Numera o quadro seguinte daquela mensagem; a contagem pertence ao par lógico, e não ao dispositivo
<code>Camada.descer</code> e <code>Camada.subir</code>	unidade e contexto	unidades e eventos	Declaram a interface comum das sete camadas; a classe base não as implementa
<code>Camada._meu_cabecalho</code>	unidade	Cabecalho ou nada	Via de leitura de cabeçalho: consulta sempre o número da

Elemento	Recebe	Devolve	O que faz
			própria camada, e ignora finalizadores
Camada._evento	contexto, ação, descrição, unidade	Evento	Monta a linha do registro, com o passo seguinte e o tamanho em octetos; ação fora de ACOES é erro
octetos_do_quadro	quadro	octetos	Reúne os octetos cobertos pela soma: os cabeçalhos, como octetos opacos, e os dados; o finalizador fica de fora
soma_de_verificacao	octetos	inteiro	Soma simples dos octetos, truncada em 32 bits
Aplicacao.descer (L7)	mensagem	mensagem e o evento GERA	Marca a unidade como de camada 7 e identifica o processo que envia
Aplicacao.subir	mensagem	mensagem e o evento ENTREGA	Entrega a mensagem ao processo de destino
Apresentacao.descer (L6)	mensagem em texto	mensagem em octetos cifrados e o evento CODIFICA	Converte o texto em octetos UTF-8 e cifra com ou-exclusivo de chave fixa, sem alterar o tamanho
Apresentacao.subir	mensagem cifrada	mensagem em texto e o evento DECIFRA	Desfaz o ou-exclusivo e decodifica os octetos
Sessao.descer (L5)	mensagem	mensagem com o cabeçalho de sessão e o evento ABRE	Abre a sessão e acrescenta o cabeçalho, uma única vez, antes da segmentação
Sessao.subir	mensagem	mensagem sem o cabeçalho de sessão e o evento ENTREGA	Lê o identificador de sessão e remove o cabeçalho
Transporte.descer (L4)	mensagem	um ou vários segmentos, um evento SEGMENTA por segmento	Numera o par de portas e, quando a unidade excede o limiar, divide os dados no limite por segmento

Elemento	Recebe	Devolve	O que faz
Transporte.subir	segmento	nada enquanto faltam segmentos; a mensagem remontada quando todos chegaram	Guarda os segmentos por par de portas, remonta em ordem e associa a porta de destino ao processo que a escuta; sem processo, descarta (eventos RECEBE, REMONTA ou DESCARTA)
Rede.descer (L3)	segmento ou pacote	pacote e os eventos ENCAPSULA e ROTEIA, ou nada e o evento DESCARTA	Insere o par de endereços lógicos na origem, escolhe interface de saída e próximo salto e os grava no contexto; sem rota, descarta o pacote
Rede._escolher_salto	endereço de destino e contexto	interface de saída, vizinho e descrição	Entrega direta quando o destino está na rede de uma interface local; senão, o roteador consulta a tabela de encaminhamento e o computador entrega ao roteador da sua rede
Rede.subir	pacote	pacote e o evento ENTREGA, o pacote intacto sem evento, ou nada e o evento DESCARTA	Entrega à camada 4 quando o destino é do próprio dispositivo; num roteador devolve o pacote para a descida, que registra o salto
Enlace.descer (L2)	pacote	quadro e o evento ENQUADRA	Traduz o próximo salto indicado pela camada 3 no endereço físico do vizinho na mesma rede, numera o quadro e acrescenta cabeçalho e finalizador com a soma
Enlace._meu_finalizador	quadro	Cabecalho ou nada	Lê o finalizador da própria camada, que _meu_cabecalho não alcança; também preso a self.numero
Enlace.subir	quadro	pacote e o evento DESENQUADRA, ou	Recalcula a soma e a compara com a recebida;

Elemento	Recebe	Devolve	O que faz
		nada e o evento DESCARTA	divergindo, descarta o quadro; senão remove cabeçalho e finalizador
<code>Fisica.descer</code> (L1)	quadro	bits e o evento TRANSMITE	Converte o quadro em bits e o transmite
<code>Fisica.subir</code>	bits	quadro e o evento RECEBE	Reagrupa os bits no quadro

dispositivos.py

Elemento	Recebe	Devolve	O que faz
<code>Dispositivo</code>	nome, topologia e os contadores da execução	—	Monta a pilha de camadas da classe e o contexto que elas compartilham
<code>Dispositivo.numeros_das_camadas</code>	—	lista de inteiros	Números das camadas montadas, da mais alta para a mais baixa
<code>Dispositivo.descer</code>	unidades	unidades que saíram da camada 1 e a lista de passos	Leva as unidades da camada mais alta da pilha até a camada 1
<code>Dispositivo.subir</code>	unidades	unidades que chegaram ao topo e a lista de passos	Leva as unidades da camada 1 até a camada mais alta da pilha
<code>Dispositivo.receber</code>	quadro	entregues, quadros a transmitir e passos	Trata um quadro vindo do meio físico; cada tipo de dispositivo define o seu
<code>Computador</code>	—	—	Pilha das sete camadas, da aplicação à física
<code>Computador.escutar</code>	porta e nome do processo	—	Associa um processo a uma porta, para a demultiplexação da camada 4

Elemento	Recebe	Devolve	O que faz
<code>Computador.enviar</code>	mensagem, endereço lógico de destino, par de portas e processo	quadros e passos	Grava o envio no contexto e leva a mensagem da camada 7 à camada 1
<code>Computador.receber</code>	quadro	mensagens entregues, nenhum quadro a transmitir e passos	Sobe o quadro pelas sete camadas
<code>Roteador</code>	—	—	Pilha de três camadas: rede, enlace e física
<code>Roteador.receber</code>	quadro	nenhuma entrega, os quadros novos e os passos	Sobe até a camada 3, que decide a rota, e desce de novo; um pacote endereçado ao próprio roteador não segue adiante
<code>criar_dispositivo</code>	nome, topologia e os contadores	<code>Computador</code> ou <code>Roteador</code>	Monta a pilha certa para o nome, conforme a topologia
<code>_parear</code>	eventos, unidades produzidas e unidade recebida	lista de pares (evento, unidade)	Associa cada evento à unidade que ele descreve, que é o que a interface desenha

`simulador.py`

Elemento	Recebe	Devolve	O que faz
<code>Falhas</code>	enlace derrubado, enlace do bit invertido, posição do bit e destino inalcançável	—	Descreve as falhas injetadas em uma execução; sem argumentos, nenhuma
<code>Fluxo</code>	origem, destino, par de processos, par de portas e mensagem	—	Uma mensagem de um processo de origem para um processo de destino

Elemento	Recebe	Devolve	O que faz
Cenario	código, nome, fluxos e falhas	—	Um caso de validação
Transmissao	emissor, receptor e quadro	—	Um quadro posto em um enlace, na forma em que o emissor o enviou
Entrega	dispositivo, processo, sessão, portas, lógicos e mensagem	—	Uma mensagem que chegou à camada 7 de um destino
Resumo	quadros, octetos da mensagem, úteis, transmitidos, eficiência e sobrecarga	—	O quadro exibido ao fim de cada execução
inverter_bit	quadro e posição do bit	quadro novo	Inverte um bit dos dados, como faria um ruído, deixando intacta a soma do finalizador
Simulacao	topologia e cenário	—	Monta os dispositivos, faz o papel do meio físico e acumula o registro; trabalha sobre uma cópia da topologia, com os próprios enlaces derrubados
Simulacao.passo	—	Evento ou nada	Avança um evento, registra-o e guarda em unidade a unidade de dados que ele descreve
Simulacao.terminou	—	booleano	Informa se ainda há algum evento por vir
Simulacao.executar	—	lista de eventos	Executa o que falta e devolve todos os eventos da execução
Simulacao.dispositivo	nome	Computador ou Roteador	Devolve o dispositivo, criando-o na primeira

Elemento	Recebe	Devolve	O que faz
			vez com os contadores da execução
<code>Simulacao.quadros</code>	—	lista de unidades	Os quadros postos no meio, na ordem de transmissão
<code>Simulacao.resumo</code>	—	Resumo	Conta quadros e octetos e calcula eficiência e sobrecarga
<code>Simulacao._percorrer</code>	—	sequência de pares (evento, unidade)	Envia os fluxos, intercala as rajadas e faz cada quadro avançar um enlace por vez
<code>Simulacao._transmitir</code>	emissor e quadros	itens a enfileirar e passos	Põe os quadros no meio; um enlace fora de serviço produz o descarte, e o enlace escolhido corrompe o primeiro quadro que o atravessa
<code>gerar_registros_dos_cenarios</code>	topologia e, opcionalmente, a pasta	lista de (código, caminho, número de eventos)	Regera os sete arquivos de registros/ a partir de uma execução limpa de cada cenário
CENARIOS	—	dicionário de Cenario	Os sete cenários de validação, de E1 a E7

`registro.py`

Elemento	Recebe	Devolve	O que faz
ACOES	—	conjunto de nomes	Os catorze nomes de ação aceitos no registro
Evento	passo, dispositivo, camada, ação, descrição e tamanho	—	Uma linha do registro: o que uma camada fez em um dispositivo

Elemento	Recebe	Devolve	O que faz
<code>formatar</code>	um <code>Evento</code>	texto	Monta a linha oficial, com o passo em três dígitos e o tamanho em octetos alinhado à direita
<code>Registro</code>	eventos iniciais, opcionais	—	Acumula os eventos de uma simulação, na ordem em que ocorreram
<code>Registro.registrar</code>	lista de eventos	—	Acrescenta eventos ao fim do registro
<code>Registro.linhas</code>	—	lista de textos	Formata todos os eventos acumulados
<code>Registro.salvar_em_arquivo</code>	caminho	—	Grava o registro formatado em UTF-8, uma linha por evento

`visual.py`

Elemento	Recebe	Devolve	O que faz
<code>executar</code>	a função que carrega a topologia	—	Abre a janela e só retorna quando o usuário a fecha; sem tela gráfica levanta <code>JanelaIndisponivel</code>
<code>Janela</code>	a raiz do tkinter e a função de carga	—	Monta a barra de controle, a barra lateral, o mapa, as pilhas, a unidade de dados, os endereços e o registro, e conduz a execução
<code>Janela.passo</code>	—	booleano	Avança um evento da simulação, incorpora-o ao andamento e redesenha
<code>Janela.executar</code> e <code>Janela.pausar</code>	—	—	Iniciam e interrompem a execução contínua, no intervalo da velocidade escolhida
<code>Janela.ate_o_fim</code>	—	—	Percorre de uma vez os eventos restantes
<code>Janela.reiniciar</code>	—	—	Descarta a execução corrente e prepara outra com a configuração atual

Elemento	Recebe	Devolve	O que faz
<code>Janela.salvar_registro</code>	—	—	Grava o registro em arquivo escolhido pelo usuário, a partir da pasta do programa
<code>Janela.regerar_registros</code>	—	—	Regera a pasta <code>registros/</code> com os sete cenários
<code>Janela.recarregar_topologia</code>	—	—	Relê o arquivo de topologia e refaz a configuração
<code>Escolhas</code>	os campos da barra lateral, ainda como texto	—	O que o usuário preencheu: cenário de partida, origem, destino, mensagem e falhas
<code>ConfiguracaoInvalida</code>	mensagem e nome do campo	—	Campo que impede a execução; a janela usa o nome para destacar o campo
<code>montar_cenario</code>	topologia e <code>Escolhas</code>	<code>Cenario</code> e um booleano	Transforma as escolhas em cenário e informa se ele difere da tabela de validação
<code>resolver_destino</code>	topologia, texto e origem	nome do computador	Aceita o nome ou o endereço lógico; recusa roteadores, endereços sem dono e a própria origem
<code>validar_inalcancavel</code>	topologia e texto	endereço	Exige um endereço bem formado e sem dono na topologia
<code>endereco_bem_formado</code>	texto	booleano	Confere quatro números de 0 a 255 separados por ponto
<code>dispositivos_do_caminho</code>	topologia e cenário	lista de nomes e o total de passos	Dispositivos cujas pilhas aparecem, na ordem da tela, obtidos de um ensaio da mesma execução
<code>rotulos_de_enlace</code>	topologia	rótulo de cada enlace e o par de dispositivos	Monta as listas de enlace das falhas
<code>posicoes_no_mapa</code>	topologia, largura, altura e margens	coordenada de cada dispositivo	Põe os roteadores em círculo e os computadores

Elemento	Recebe	Devolve	O que faz
			do lado de fora do roteador a que se ligam
Andamento	topologia	—	Estado da tela derivado dos eventos já ocorridos: alcançados, enlaces percorridos, quadro em trânsito, descartes e saltos
Andamento.registrar	evento e unidade	—	Incorpora um passo; o enlace vem do par de endereços físicos da unidade, nunca do texto da descrição
Tipografia	—	—	As fontes da janela, todas presas ao mesmo fator de escala, e as medidas de desenho que crescem com o texto
Mapa.desenhar	topologia, falhas, andamento e erro	—	Desenha os dispositivos, os enlaces com interface e custo, o caminho percorrido e a legenda
Pilhas.desenhar	dispositivos, papéis, roteadores, andamento, modo e aviso	—	Desenha uma pilha por dispositivo, com a camada ativa em destaque, agrupada em OSI ou em TCP/IP
Blocos.desenhar	unidade e evento	—	Desenha os cabeçalhos, os dados e o finalizador com os tamanhos, e os campos de cada cabeçalho
caber e quebrar	texto, fonte e largura	texto ou linhas	Cortam com reticências e quebram em linhas que cabem na largura disponível

6. Parâmetros configuráveis

Os parâmetros abaixo ficam no bloco `convencoes` do arquivo `topologia.json` e são lidos em tempo de execução, por quatro chaves: `cabecalhos`, `finalizador_camada_2`, `limiar_segmentacao_octetos` e `limite_segmento_octetos`. Nenhum deles está escrito no meio do código — trocar um valor no arquivo muda os números produzidos sem tocar em nenhum módulo.

Parâmetro	Faixa válida	Padrão	Efeito
Cabeçalho da camada 5	inteiro não negativo	4 octetos	Entra uma única vez, antes da segmentação
Cabeçalho da camada 4	inteiro não negativo	8 octetos	Entra em cada segmento
Cabeçalho da camada 3	inteiro não negativo	20 octetos	Entra em cada pacote
Cabeçalho da camada 2	inteiro não negativo	14 octetos	Entra em cada quadro, em cada enlace
Finalizador da camada 2	inteiro não negativo	4 octetos	Acompanha o cabeçalho da camada 2
Limiar de segmentação	maior que o limite do segmento	64 octetos	Acima dele a camada 4 segmenta
Limite por segmento	maior que zero	40 octetos	Octetos de dados em cada segmento

O critério de desempate entre caminhos de mesmo custo não é um parâmetro do arquivo: ele é aplicado por construção, na ordenação do código, e está descrito na seção 8. Um valor no `topologia.json` não o alteraria.

Fora do arquivo de topologia:

Parâmetro	Faixa válida	Padrão
Caminho do arquivo de topologia	ao lado do programa ou cópia embutida	<code>topologia.json</code> ao lado do programa; na falta dele, a cópia embutida no executável
Velocidade de reprodução	Lenta, Normal ou Rápida	Normal, 600 ms (Lenta 1500 ms, Rápida 150 ms)
Tamanho do texto	Normal, Grande ou Projetor	Normal
Modo da pilha	OSI ou TCP/IP	OSI

Os três últimos são escolhidos na barra de controle da janela e valem apenas para a apresentação: nenhum deles altera os eventos produzidos pelo núcleo. A busca do arquivo de topologia, em duas etapas, está detalhada na seção 9.

7. Funcionamento interno

A descida na origem

A camada 7 gera a mensagem e identifica o processo pelo nome. A camada 6 converte o texto em octetos e cifra o conteúdo, sem alterar o número de octetos. A camada 5 abre a sessão e acrescenta seu

cabeçalho de 4 octetos. A camada 4 recebe a mensagem já com o cabeçalho de sessão, numera as portas de origem e destino e segmenta quando necessário.

A camada 3 insere o par de endereços lógicos e decide o encaminhamento: se o destino compartilha o prefixo de rede da origem, a entrega é direta; caso contrário o pacote é enviado ao roteador da rede local. A camada 2 insere o par de endereços físicos do salto, delimita o quadro e calcula a verificação de erro. A camada 1 converte o quadro em bits e o transporta.

O que acontece em cada salto

O quadro chega à camada 1 do roteador e sobe para a camada 2, que confere a verificação de erro, remove o cabeçalho e o finalizador e entrega o pacote à camada 3. O quadro recebido é então descartado: ele não é reescrito nem reaproveitado.

A camada 3 consulta a tabela de encaminhamento com o endereço lógico de destino, que não mudou, e determina o próximo salto e a interface de saída. Registra essa decisão no contexto e devolve o pacote para a descida.

A camada 2 lê a decisão da camada 3, traduz o nome do próximo salto no endereço físico da interface do vizinho que está na mesma rede da interface de saída, e constrói um quadro novo, com número próprio. A camada 2 não sabe para onde o pacote vai, e a camada 3 não sabe por qual meio ele chegou.

A consulta à tabela de encaminhamento

As tabelas são calculadas a partir dos custos dos enlaces, por busca do caminho de menor custo entre roteadores. Para cada rede da topologia, a tabela registra o prefixo, o próximo salto, a interface de saída e o custo total. Redes diretamente conectadas ao roteador aparecem com custo zero e entrega direta.

Enlaces retirados de serviço ficam fora do cálculo, de modo que derrubar um enlace altera as tabelas sem que nenhuma outra parte do programa precise ser avisada.

Quando o endereço de destino não pertence a nenhum prefixo conhecido, a consulta não devolve rota. A camada 3 descarta o pacote e registra o descarte. Isso ocorre no primeiro roteador que não encontra rota, e não na origem.

A remontagem no destino

A camada 4 do destino acumula os segmentos recebidos e só entrega à camada 5 depois que todos chegaram, remontando-os na ordem correta. Enquanto aguarda, devolve a tupla vazia: nada sobe.

Quando dois fluxos chegam ao mesmo processo de destino, a camada 4 os separa pelo par de portas e entrega cada um à sua sessão. Os segmentos de fluxos distintos podem chegar intercalados sem que se misturem.

O quadro corrompido

A camada 2 do receptor recalcula a verificação de erro sobre o quadro recebido e a compara com a que veio no finalizador. Divergindo, o quadro é descartado e o descarte é registrado. Nenhuma camada superior é acionada e não há retransmissão: o registro do dispositivo receptor não contém nenhuma linha de camada 3.

8. Convenções de simulação

As decisões abaixo determinam os números produzidos. Sem elas, a mesma mensagem gera resultados diferentes.

Tamanho dos cabeçalhos

Cabeçalhos de tamanho fixo: 4 octetos na camada 5, 8 na camada 4, 20 na camada 3, e 14 de cabeçalho mais 4 de finalizador na camada 2. As camadas 6 e 7 não acrescentam octetos. A cifragem da camada 6 não altera o número de octetos da mensagem.

O cabeçalho de sessão entra uma única vez

A camada 5 acrescenta seu cabeçalho de 4 octetos à mensagem inteira, antes da segmentação. A camada 4 segmenta os dados que recebe da camada 5, com o cabeçalho de sessão já incluído neles.

Uma mensagem de 100 octetos chega à camada 4 com 104 e é dividida em 40, 40 e 24. Tratar o cabeçalho de sessão como parte de cada segmento produziria 40, 40 e 28, e mudaria todos os totais do cenário E7.

O limiar de segmentação é distinto do tamanho do segmento

A segmentação ocorre quando a unidade recebida da camada 5 excede **64 octetos**. Ocorrendo, ela é dividida em segmentos de no máximo **40 octetos** de dados, e o último segmento pode ser menor.

Os dois valores são distintos por necessidade aritmética. A mensagem de referência de 42 octetos chega à camada 4 com 46. Aplicando o limite de 40 octetos diretamente, ela seria dividida em 40 e 6, produzindo dois segmentos, e o cenário E2 transmitiria oito quadros em vez de quatro. Os valores de referência da especificação exigem um único segmento nesse caso — o registro de referência registra **segmento 1 de 1** — e três segmentos de 40, 40 e 24 para a mensagem de 100 octetos, que chega com 104.

Um único número não satisfaz as duas exigências. O limiar de 64 octetos é o valor adotado; qualquer valor entre 47 e 104 reproduz os mesmos resultados nos sete cenários.

A eficiência usa os octetos entregues

A eficiência é a razão entre os octetos efetivamente entregues à camada 7 do destino e os octetos transmitidos somando todos os enlaces. O numerador conta o que chegou ao processo de destino, sem o cabeçalho de sessão: no cenário E7, 100 octetos, e não 104.

A escolha do numerador é o que distingue os cenários em que a mensagem se perde. Não havendo entrega, o numerador é zero e a eficiência do cenário também, ainda que octetos tenham trafegado até o ponto do descarte — é o que ocorre em E5 e E6.

Desempate entre caminhos de mesmo custo

O custo de um caminho é a soma dos custos dos enlaces que o compõem. Empates são resolvidos pelo menor identificador de roteador. O critério é aplicado por construção: os vizinhos são ordenados por custo e nome antes de serem considerados, e a seleção do próximo nó ordena por custo e nome. A escolha nunca depende da ordem em que os elementos aparecem no arquivo de topologia.

Numeração dos quadros

Os quadros são numerados Q1, Q2 e assim por diante, na ordem de transmissão, reiniciando a cada mensagem. Cada enlace percorrido produz um quadro distinto.

O quadro descartado por erro

O quadro com erro é descartado pela camada 2 do receptor, sem notificação às camadas superiores e sem retransmissão. Os octetos já transmitidos até o ponto do descarte contam no total transmitido; como não há entrega, a eficiência do cenário é zero.

Verificação de erro

A verificação de erro é uma soma calculada sobre os cabeçalhos e os dados do quadro, excluindo o próprio finalizador. Cobrir também os cabeçalhos é necessário porque a alteração de um bit pode recair sobre um campo de cabeçalho, e não apenas sobre os dados.

Cifragem da camada 6

A cifragem é uma operação de ou-exclusivo com chave fixa, aplicada octeto a octeto. A escolha atende às duas exigências do escopo: o conteúdo é ilegível entre a camada 6 da origem e a camada 6 do destino, e o número de octetos não se altera. Nenhum roteador e nenhuma camada intermediária tem acesso ao texto claro.

Numeração dos passos

Os passos do registro são numerados sequencialmente a partir de 001, um passo por ação de camada, na ordem em que ocorrem.

9. Dependências e ambiente

O programa é escrito em Python 3.13 e utiliza exclusivamente a biblioteca padrão. A interface gráfica usa `tkinter`, que acompanha a instalação oficial do Python. Não há nenhuma biblioteca externa em tempo de execução.

O executável é gerado com PyInstaller, que é dependência apenas de desenvolvimento e não participa da execução:

```
pyinstaller Simulador0SI.spec
```

A receita fica no `Simulador0SI.spec`, versionado no repositório, e não na linha de comando. O motivo é o `--add-data`: seu separador muda entre sistemas (`topologia.json:.` no macOS e no Linux, `topologia.json;.` no Windows), de modo que um comando único não serviria aos dois. Dentro do spec, `datas` recebe o par `('topologia.json', '.')` já separado, e o PyInstaller monta o caminho com o separador do sistema em que roda. O spec também fixa `console=True`, que é o que permite ao modo texto assumir e mostrar o menu quando o `tkinter` não consegue abrir a janela.

O empacotamento produz um arquivo único, que se descompacta em uma pasta temporária a cada execução. Essa pasta deixa de existir quando o programa encerra, o que traz duas consequências tratadas pelo módulo `recursos.py`.

O arquivo de topologia é procurado ao lado do executável e, somente se não for encontrado ali, a cópia embutida pelo spec é utilizada. O caminho é derivado do executável, e não do módulo em execução, porque este último aponta para a pasta temporária.

Pelo mesmo motivo, o registro de eventos salvo pelo programa é gravado na pasta do executável, nunca na pasta de descompactação.

Anexo: conferência dos valores de referência

A verificação dos números é automatizada e pode ser repetida a partir do código-fonte:

Arquivo	O que confere
<code>testes/teste_convencoes.py</code>	Octetos transmitidos e eficiência dos sete cenários
<code>testes/teste_rede.py</code>	Tabelas de encaminhamento e escolha de rota
<code>testes/teste_camadas.py</code>	Encapsulamento, segmentação, remontagem e descarte
<code>testes/teste_cenarios.py</code>	Os sete cenários completos, pela simulação